SQL QUERY IMPLEMENTATION

Introduction

SQL (Structured Query Language) is used to store, retrieve, and manage data in a relational database. In an E-Commerce system, queries help search products, view customer details, and generate reports for business decisions. This experiment uses the CUSTOMER, PRODUCT, CATEGORY, and INVENTORY tables.

1.Perform SELECT, WHERE, ORDER BY, and DISTINCT queries.

1.1 Display All Customers (SELECT)

The SELECT statement with * retrieves every column and row from the CUSTOMER table

```
SQL> -- 1. Display all customers
SQL> SELECT * FROM CUSTOMER;
```

1.2 Display Customers Aged Above 19 (WHERE)

The WHERE clause filters rows by a condition. Only customers whose age is greater than 19 are displayed**.**

```
SQL>
SQL> -- 2. Display customers aged above 19
SQL> SELECT * FROM CUSTOMER
  2  WHERE AGE > 19;
```

1.3 Display Customers in Alphabetical Order (ORDER BY)

The ORDER BY clause sorts the result. ASC arranges the customer names from A to Z.

```
SQL>
SQL> -- 3. Display customers in alphabetical order
SQL> SELECT * FROM CUSTOMER
  2  ORDER BY NAME ASC;
```

1.4 Display Unique Customer Addresses (DISTINCT)

The DISTINCT keyword removes duplicate values, so each city appears only once.

```
SQL>
SQL> -- 4. Display unique customer addresses
SQL> SELECT DISTINCT ADDRESS
  2  FROM CUSTOMER;
```

2. Search Products Based on Price, Category, and Availability

2.1 Search Products by Price

Products are searched using a price condition. This query shows products priced below 1000.

```
SQL> -- Display all customer details
SQL> SELECT * FROM CUSTOMER;
```

2.2 Search Products by Category

Products are filtered using CATEGORY_ID, so only products of the selected category are displayed

```
SQL>
SQL> -- Display product names and prices
SQL> SELECT PRODUCT_NAME, PRICE
  2  FROM PRODUCT;
```

2.3 Search Available Products

Availability is checked using the STOCK column. Only products with stock greater than 0 are displayed.

```
SQL>
SQL> -- Search available products
SQL> SELECT * FROM PRODUCT
  2  WHERE STOCK > 0;
```

3. Retrieve Customer and Product Information

3.1 Display Customer Information

This query retrieves the ID, name, age, address, and salary of customers.

```
SQL> -- Display customer information
SQL> SELECT ID, NAME, AGE, ADDRESS, SALARY
  2  FROM CUSTOMER;
```

3.2 Display Product Information

This query retrieves product details such as name, brand, price, stock, and category.

```
SQL>
SQL> -- Display product names and prices
SQL> SELECT PRODUCT_NAME, PRICE
  2  FROM PRODUCT;
```

3.3 Table Structures

The DESC command shows the column names, data types, and constraints of the CUSTOMER, PRODUCT, CATEGORY, and INVENTORY tables.

```
SQL> -- Products priced between Rs. 1000 and Rs. 3000
SQL> SELECT * FROM PRODUCT
  2  WHERE PRICE BETWEEN 1000 AND 3000;
```

4. Apply Filtering Conditions

4.1 Products within a Price Range

The BETWEEN operator selects products whose price falls within a given range (100 to 1000).

```
SQL> -- Products within a price range
SQL> SELECT * FROM PRODUCT
  2  WHERE PRICE BETWEEN 100 AND 1000;

no rows selected
```

4.2 Products Matching Multiple Conditions

The AND operator combines conditions. Only products with price above 500 and stock available are displayed.

```
SQL>
SQL> -- Customers from Chennai
SQL> SELECT * FROM CUSTOMER
  2  WHERE ADDRESS = 'Chennai';
```

4.3 Customers from Chennai

The WHERE clause on the ADDRESS column displays only customers who live in Chennai**.**

```
SQL> -- Electronics products below Rs. 2000
SQL> SELECT * FROM PRODUCT
  2  WHERE CATEGORY = 'Electronics'
  3  AND PRICE < 2000;
```

5. Generate Basic Business Reports

5.1 Total Number of Customers

COUNT gives the total number of customer records.

```
SQL> -- Total number of products
SQL> SELECT COUNT(*) AS TOTAL_PRODUCTS
  2  FROM PRODUCT;

TOTAL_PRODUCTS
--------------
             6
```

5.2 Average Customer Salary

AVG calculates the average salary of all customers.

```
SQL>
SQL> -- Total stock quantity
SQL> SELECT SUM(STOCK_QUANTITY) AS TOTAL_STOCK
  2  FROM PRODUCT;
```

5.3 Highest-Priced Product

MAX returns the highest product price.

```
SQL> -- Category-wise product count
SQL> SELECT CATEGORY, COUNT(*) AS TOTAL_PRODUCTS
  2  FROM PRODUCT
  3  GROUP BY CATEGORY;
GROUP BY CATEGORY
```

5.4 Total Number of Products

COUNT gives the total number of products available.

```
SQL>
SQL> -- Average product price
SQL> SELECT AVG(PRICE) AS AVERAGE_PRICE
  2  FROM PRODUCT;

AVERAGE_PRICE
-------------
   10915.8333
```

Result

SQL queries using SELECT, WHERE, ORDER BY, and DISTINCT were executed successfully. Products were searched by price, category, and availability, customer and product information was retrieved, filtering conditions were applied, and basic business reports were generated.